

INSTITUTO INFNET

Pós-Graduação em Engenharia de Dados e Big Data

PROJETO 2

Aluno: Willon Ferreira da Silva

Data: Outubro de 2026

1. Introdução

Neste trabalho, construí um lakehouse no Databricks seguindo a arquitetura Medallion (Bronze, Silver e Gold) para uma base de matrículas de alunos. Os dados chegam de uma fonte externa (endpoint HTTP que retorna CSV), são capturados pelo Auto Loader, validados com expectations, atualizados com CDC (SCD tipos 1 e 2) e publicados em uma camada Gold modelada em esquema estrela.

O pipeline foi feito com o **Lakeflow Declarative Pipelines**, novo nome do Delta Live Tables (DLT), usando a API Python `from pyspark import pipelines as dp`. Todas as tabelas são Delta e ficam registradas no Unity Catalog, no catálogo `projeto2`.

1.1 Objetivos

- Ingerir arquivos CSV de forma incremental e escalável com Auto Loader.
- Isolar registros inválidos em uma tabela de quarentena, sem perder dados.
- Manter o histórico dos alunos com CDC e SCD tipo 2 na Silver.
- Publicar uma Gold pronta para consumo analítico, com controle de acesso e mascaramento de dados pessoais.
- Otimizar o layout físico das tabelas (particionamento, Z-Order e compactação) e monitorar a execução.

1.2 Dataset

A base possui uma linha por matrícula. As colunas são:

Coluna	Descrição	Origem
<code>matricula</code>	Identificador da matrícula (chave do CDC)	Fonte
<code>nome, email, cpf, data_nascimento</code>	Dados pessoais do aluno	Fonte
<code>curso, periodo, turma</code>	Dados acadêmicos da matrícula	Fonte
<code>nota_final, frequencia</code>	Métricas de desempenho	Fonte
<code>status</code>	Situação: ativo, trancado, formado ou jubilado	Fonte
<code>bolsa</code>	Indica se o aluno é bolsista	Fonte
<code>cidade, uf</code>	Localidade do aluno	Fonte
<code>data_matricula</code>	Data em que a matrícula foi feita	Fonte
<code>_rescued_data</code>	Dados que não se encaixaram no schema (Auto Loader)	Bronze
<code>ingestion_timestamp</code>	Momento da ingestão; usado como sequência no CDC	Bronze

1.3 Tecnologias

Componente	Uso no projeto
Unity Catalog	Catálogo <code>projeto2</code> , schemas <code>bronze</code> , <code>silver</code> e <code>gold</code> , volumes, permissões, tags e máscaras
Delta Lake	Formato de todas as tabelas (ACID, histórico e Time Travel)
Auto Loader (<code>cloudFiles</code>)	Captura incremental dos CSVs que chegam no volume <code>raw_csv</code>
Lakeflow Declarative Pipelines (DLT)	Streaming tables, materialized views, expectations e AUTO CDC

Componente	Uso no projeto
Lakeflow Jobs	Orquestração das etapas (ingestão → bronze → silver → gold)
Compute Serverless	Execução dos pipelines e das queries

2. Arquitetura da solução

A solução segue o fluxo evolutivo do lakehouse. Cada camada fica em um schema próprio do catálogo **projeto2**, e cada camada tem o seu pipeline.

Fluxo de dados

```

API externa (CSV via HTTP)
|
v
[Notebook 02-ingestao] -> /Volumes/projeto2/bronze/raw_csv/AAAAMMDD_HHMMSS.csv
|
v
[Pipeline BRONZE] Auto Loader
projeto2.bronze.projeto2_bronze (streaming table)
|
v
[Pipeline SILVER] Expectations + CDC
projeto2.silver.projeto2_quality (streaming table)
|-- projeto2.silver.projeto2_quarantine (registros reprovados)
'-- projeto2.silver.projeto2_silver_quality (registros aprovados)
    |-- projeto2.silver.dim_aluno_scd1 (AUTO CDC - SCD 1)
    '-- projeto2.silver.dim_aluno_scd2 (AUTO CDC - SCD 2)
|
v
[Pipeline GOLD] Materialized views (esquema estrela)
projeto2.gold.dim_localidade | dim_curso | dim_aluno | fato_matriculas
|
v
Consumo: SQL Editor, permissões, tags de classificação e máscara de CPF
  
```

2.1 Organização do projeto

Os artefatos estão organizados no workspace conforme a estrutura abaixo. A ordem de execução segue a numeração das pastas e arquivos:

Ordem	Artefato	Tipo	Função
1	01-catalog-volumes.dbquery.ipynb	SQL	Cria catálogo, schemas e volumes
2	02-ingestao.ipynb	Notebook Python	Baixa o CSV da API e grava no volume raw_csv
3	02_projeto2_bronze/transformations/auto-loader.py	Pipeline	Camada Bronze (Auto Loader)
4	03_projeto2_silver/transformations/expectations.py	Pipeline	Expectations, quarentena e Silver validada
4	03_projeto2_silver/transformations/cdc_scd.py	Pipeline	CDC com SCD tipos 1 e 2
5	04_projeto2_gold/transformations/camada_gold.py	Pipeline	Dimensões e fato (materialized views)
6	queries.dbquery.ipynb	SQL	Permissões, governança, monitoramento e consultas na Gold

2.2 Objetos publicados

Camada	Objeto	Tipo	Descrição
Bronze	projeto2_bronze	Streaming table	Dados brutos + <code>ingestion_timestamp</code>
Silver	projeto2_quality	Streaming table	Todos os registros com expectations e flag <code>is_quarantined</code>
Silver	projeto2_quarantine	Streaming table	Registros reprovados
Silver	projeto2_silver_quality	Streaming table	Registros aprovados, particionados e com Z-Order
Silver	dim_aluno_scd1	Streaming table (AUTO CDC)	Estado atual de cada matrícula
Silver	dim_aluno_scd2	Streaming table (AUTO CDC)	Histórico de cada matrícula
Gold	dim_localidade	Materialized view	Cidades e UFs
Gold	dim_curso	Materialized view	Cursos, períodos e turmas
Gold	dim_aluno	Materialized view	Alunos com histórico (vem da SCD2)
Gold	fato_matriculas	Materialized view	Métricas agregadas de matrícula e desempenho

3. Preparação do ambiente (Unity Catalog)

O primeiro passo foi criar a estrutura no Unity Catalog: o catálogo do projeto, um schema por camada e os volumes da Bronze. O volume **raw_csv** é a zona de aterrissagem dos arquivos, e o **schema** guarda o schema inferido pelo Auto Loader.

01-catalog-volumes.dbquery.ipynb

```
-- CRIAÇÃO DO UNITY CATALOG
CREATE CATALOG IF NOT EXISTS projeto2;

-- CRIAÇÃO DOS SCHEMAS
CREATE SCHEMA IF NOT EXISTS projeto2.bronze;
CREATE SCHEMA IF NOT EXISTS projeto2.silver;
CREATE SCHEMA IF NOT EXISTS projeto2.gold;

-- CRIAÇÃO DOS VOLUMES
CREATE VOLUME IF NOT EXISTS projeto2.bronze.raw_csv;
CREATE VOLUME IF NOT EXISTS projeto2.bronze.schema;
CREATE VOLUME IF NOT EXISTS projeto2.bronze.checkpoint;
```

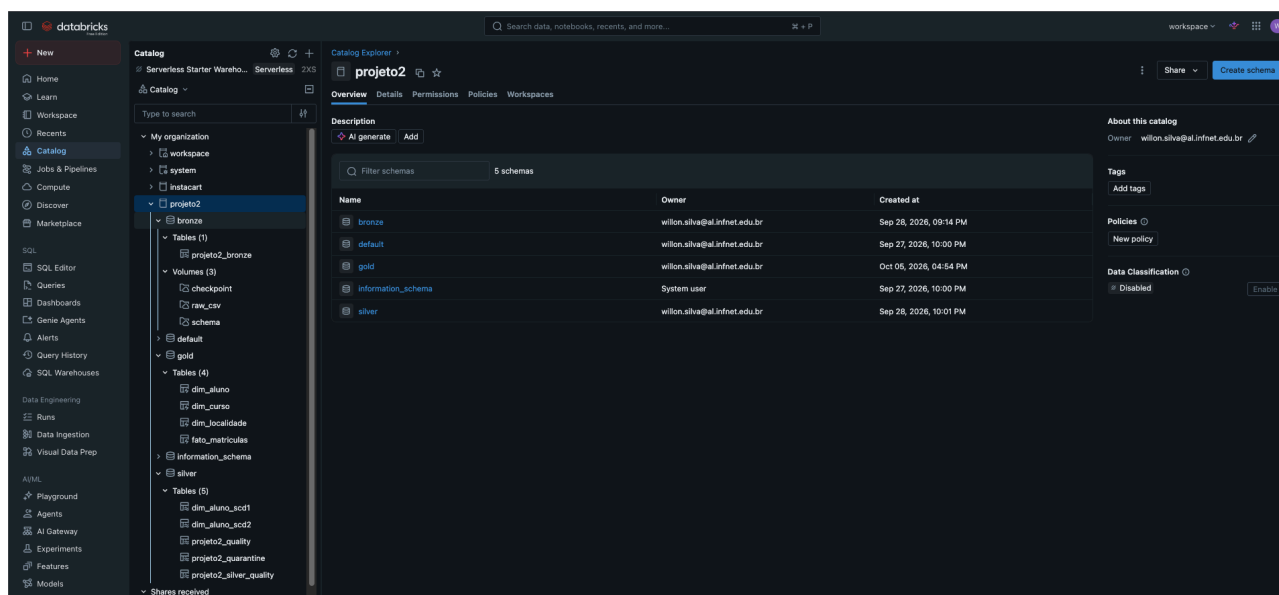


Figura 1 — Catálogo projeto2 no Catalog Explorer, com os schemas, os volumes da Bronze e as tabelas de cada camada

4. Ingestão escalável

4.1 Captura da fonte externa

O notebook `02-ingestao` faz um GET na API, verifica se o arquivo já está pronto (a API responde `"status": "updating"` enquanto gera o CSV) e grava o conteúdo no volume `raw_csv`. Cada arquivo recebe como nome a data e hora da coleta no fuso `America/Manaus`, e uso `overwrite=False` para nunca sobrescrever um arquivo já recebido. Assim o Auto Loader enxerga cada coleta como um arquivo novo.

02-ingestao.ipynb

```
import requests
from datetime import datetime
from zoneinfo import ZoneInfo

URL = "https://silos-deploy.onrender.com/d/Ynot115iIfpVRew2VzBPvuvR/"

RAW_PATH = "/Volumes/projeto2/bronze/raw_csv"

response = requests.get(URL, timeout=30)

response.raise_for_status()

content_type = response.headers.get("Content-Type", "")
content = response.text

print("Status:", response.status_code)
print("Content-Type:", content_type)

if '"status": "updating"' in content:
    print("Arquivo ainda não está pronto.")
    print("Nenhum CSV foi salvo.")
else:
    now = datetime.now(ZoneInfo("America/Manaus"))

    file_name = now.strftime("%Y%m%d_%H%M%S") + ".csv"

    file = f"{RAW_PATH}/{file_name}"

    dbutils.fs.put(file, content, overwrite=False)

    print(f"CSV salvo: {file}")

display(dbutils.fs.ls(RAW_PATH))
```

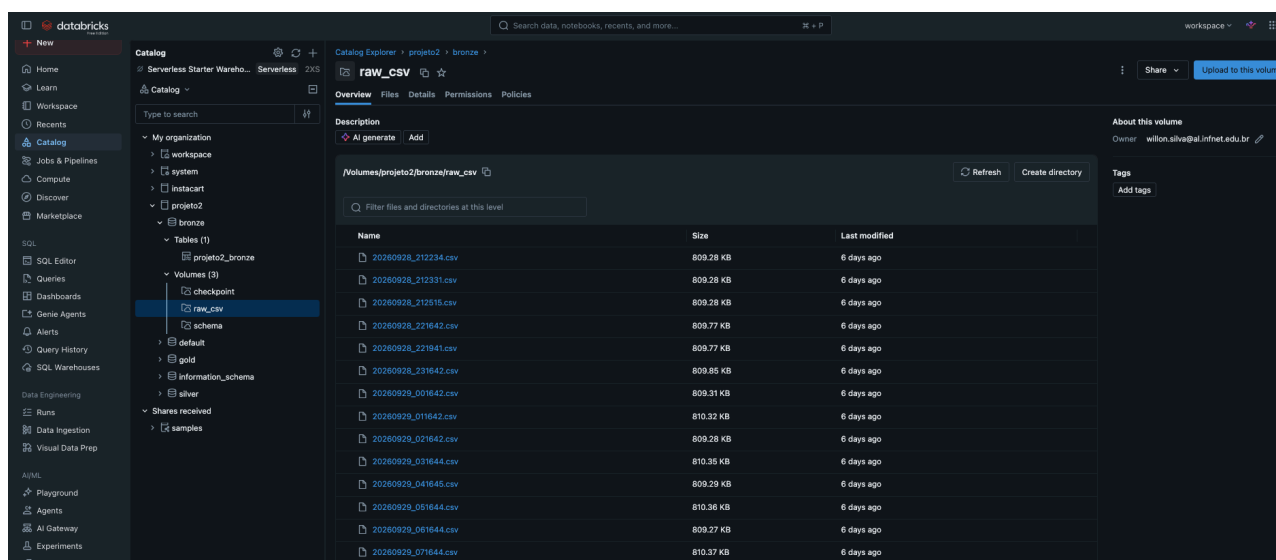


Figura 2 — Arquivos CSV gravados no volume `raw_csv`, com nome por data e hora da coleta

4.2 Camada Bronze com Auto Loader

A Bronze é uma streaming table alimentada pelo Auto Loader (`cloudFiles`). Ele processa só os arquivos novos do volume, guarda o schema em `schemaLocation` para permitir evolução de schema e coloca na coluna `_rescued_data` qualquer dado que não se encaixe no schema esperado. Também adiciono a coluna `ingestion_timestamp`, que depois é usada como sequência no CDC.

02_projeto2_bronze/transformations/autoloader.py

```
from pyspark import pipelines as dp
from pyspark.sql.functions import current_timestamp, col

@dp.table(
    name="projeto2_bronze",
    comment="Dados brutos"
)
def projeto2_bronze():

    source_path = "/Volumes/projeto2/bronze/raw_csv"
    checkpoint_path = "/Volumes/projeto2/bronze/checkpoint"
    schema_path = "/Volumes/projeto2/bronze/schema"

    return (
        spark.readStream
        .format("cloudFiles")
        .option("cloudFiles.format", "csv")
        .option("cloudFiles.schemaLocation", schema_path)
        .option("header", "true")
        .load(source_path)
        .withColumn("ingestion_timestamp", current_timestamp())
    )
```

Na última execução, o pipeline Bronze rodou em modo Triggered e escreveu 168 mil registros na `projeto2_bronze` em 12 segundos.

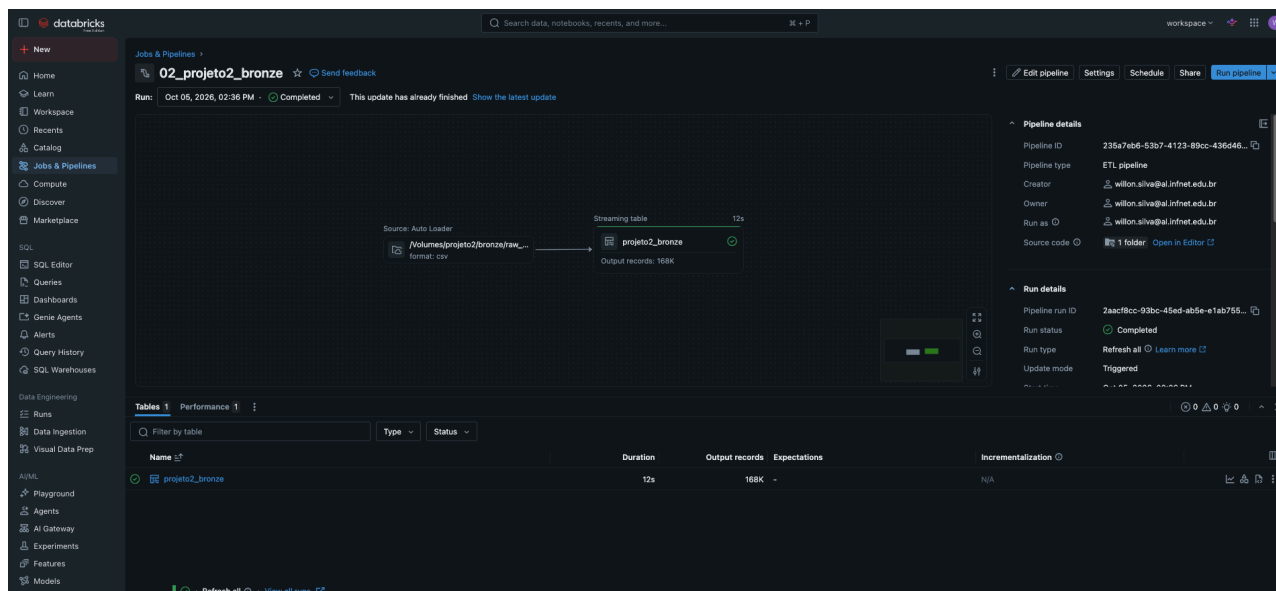


Figura 3 — Grafo do pipeline Bronze: Auto Loader lendo o volume `raw_csv` e escrevendo na streaming table `projeto2_bronze`

5. Refino transacional (camada Silver)

5.1 Validação com expectations e quarentena

Na Silver, apliquei as regras de qualidade com `@dp.expect_all`. Escolhi o modo padrão (warn): o registro não é descartado, mas o pipeline registra quantos registros passaram e falharam em cada regra. Com as mesmas regras, calculo a coluna `is_quarantined` e separo o fluxo em duas tabelas: a quarentena (reprovados) e a Silver validada (aprovados). Assim nenhum dado se perde e os registros com problema podem ser analisados e reprocessados.

Expectation	Regra
matricula_preenchida	matricula IS NOT NULL
nome_preenchido	nome IS NOT NULL
cpf_preenchido	cpf IS NOT NULL
status_valido	status IN ('ativo', 'trancado', 'formado', 'jubilado')
email_preenchido	email IS NOT NULL
curso_preenchido	curso IS NOT NULL

03_projeto2_silver/transformations/expectations.py

```
from pyspark import pipelines as dp
from pyspark.sql.functions import expr

REGRAS = {
    "matricula_preenchida":
        "matricula IS NOT NULL",

    "nome_preenchido":
        "nome IS NOT NULL",

    "cpf_preenchido":
        "cpf IS NOT NULL",

    "status_valido":
        "status IN ('ativo', 'trancado', 'formado', 'jubilado')",

    "email_preenchido":
        "email IS NOT NULL",

    "curso_preenchido":
        "curso IS NOT NULL"
}

# Condição para identificar registros que devem
# ir para a quarentena
QUARANTINE_CONDITION = (
    "NOT("
    + " AND ".join(REGRAS.values())
    + ")"
)

# =====
# QUALITY / EXPECTATIONS
# =====

@dp.table(
    name="projeto2_quality",
    comment="Registros com regras de qualidade aplicadas"
)
@dp.expect_all(REGRAS)
def projeto2_quality():
```

```

return (
    spark.readStream
        .table("projeto2.bronze.projeto2_bronze")
        .withColumn(
            "is_quarantined",
            expr(QUARANTINE_CONDITION)
        )
    )

# =====
# QUARANTINE
# =====

@dp.table(
    name="projeto2_quarantine",
    comment="Registros que não tiveram sucesso nas regras de qualidade"
)
def projeto2_quarantine():
    return (
        spark.readStream
            .table("projeto2_quality")
            .filter("is_quarantined = true")
    )

# =====
# SILVER QUALITY
# =====

@dp.table(
    name="projeto2_silver_quality",
    comment="Registros validados após as regras de qualidade",
    partition_cols=["uf", "curso"],
    table_properties={
        "pipelines.autoOptimize.zOrderCols": "cpf,cidade"
    }
)
def projeto2_silver_quality():
    return (
        spark.readStream
            .table("projeto2_quality")
            .filter("is_quarantined = false")
            .drop("is_quarantined")
    )

```

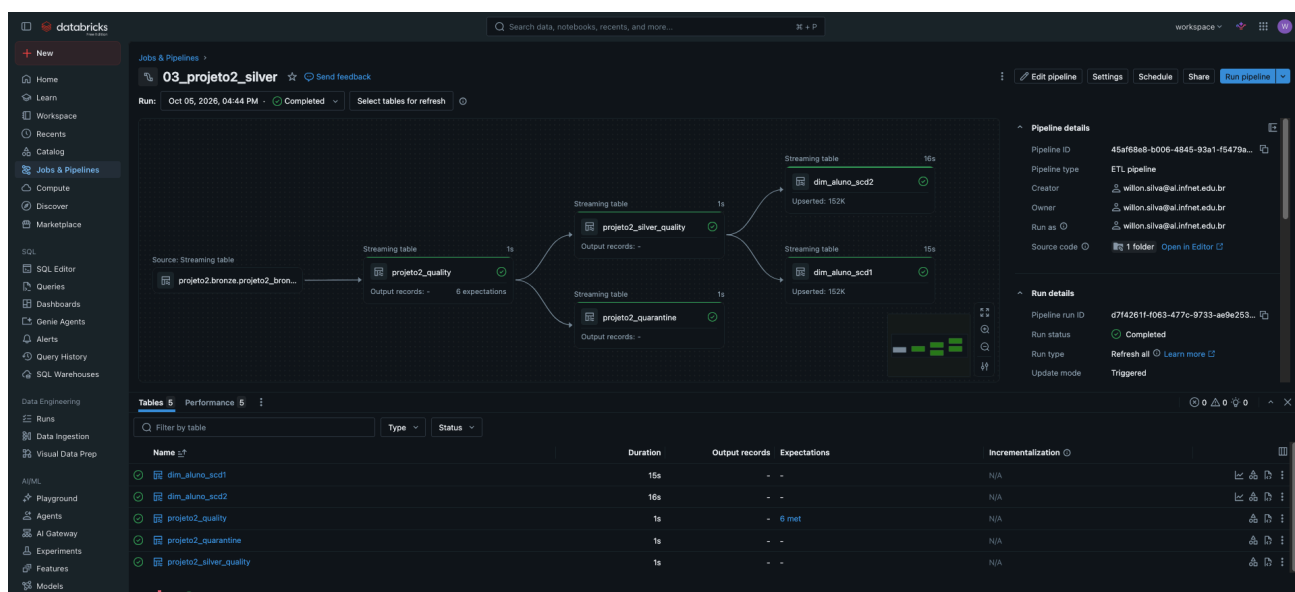


Figura 4 — Grafo do pipeline silver: projeto2_quality se dividindo em projeto2_quarantine e projeto2_silver_quality, e as tabelas SCD saindo da Silver validada

5.2 CDC com SCD tipos 1 e 2

Para manter a dimensão de alunos atualizada, usei o `create_auto_cdc_flow` (antigo `apply_changes`) a partir da Silver validada. A chave é `matricula`, porque um mesmo aluno (`cpf`) pode ter mais de uma matrícula. A ordem das mudanças é dada por `ingestion_timestamp`.

- **SCD tipo 1** (`dim_aluno_scd1`): guarda só o estado mais recente de cada matrícula. Quando chega uma alteração, a linha é sobrescrita.
- **SCD tipo 2** (`dim_aluno_scd2`): guarda o histórico. Cada alteração fecha a versão anterior e abre uma nova, controladas pelas colunas `__START_AT` e `__END_AT`. A versão atual é a que tem `__END_AT` nulo.

Retirei `_rescued_data` das duas tabelas com `except_column_list` e, na SCD2, deixei `ingestion_timestamp` fora do controle de histórico (`track_history_except_column_list`). Assim uma reingestão sem mudança real não cria versão nova.

03_projeto2_silver/transformations/cdc_scd.py

```
from pyspark import pipelines as dp
from pyspark.sql.functions import col

# =====
# IDENTIFICA O PRÓXIMO SNAPSHOT
# =====

def proximo_snapshot(latest_snapshot_version):

    df = spark.read.table(
        "projeto2_silver_quality"
    )

    snapshots = (
        df
        .select("data_matricula")
        .distinct()
        .orderBy("data_matricula")
    )

    # Primeiro snapshot
    if latest_snapshot_version is None:

        rows = (
            snapshots
            .limit(1)
            .collect()
        )

    # Próximo snapshot
    else:

        rows = (
            snapshots
            .filter(
                col("data_matricula")
                > latest_snapshot_version
            )
            .limit(1)
            .collect()
        )

    # Não existe novo snapshot
    if not rows:
        return None

    proxima_data = rows[0]["data_matricula"]

    snapshot = (
        df
        .filter(
```

```

        col("data_matricula")
        == proxima_data
    )
)

return snapshot, proxima_data

# =====
# 2. SCD TIPO 1
# =====

dp.create_streaming_table(
    name="dim_aluno_scd1",
    comment="Dimensão de alunos - SCD Tipo 1"
)

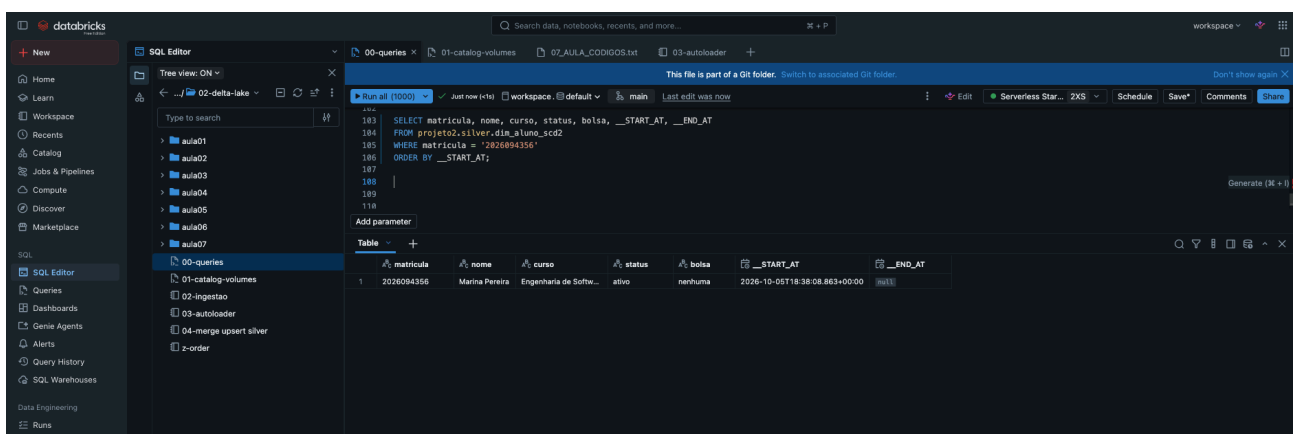
dp.create_auto_cdc_flow(
    target="dim_aluno_scd1",
    source="projeto2_silver_quality",
    keys=["matricula"],
    sequence_by="ingestion_timestamp",
    except_column_list=["_rescued_data"],
    stored_as_scd_type=1,
)

# =====
# 3. SCD TIPO 2
# =====

dp.create_streaming_table(
    name="dim_aluno_scd2",
    comment="Dimensão de alunos - SCD Tipo 2"
)

dp.create_auto_cdc_flow(
    target="dim_aluno_scd2",
    source="projeto2_silver_quality",
    keys=["matricula"],
    sequence_by="ingestion_timestamp",
    except_column_list=["_rescued_data"],
    track_history_except_column_list=["ingestion_timestamp"],
    stored_as_scd_type=2,
)

```



	matricula	nome	curso	status	bolsa	__START_AT	__END_AT
1	2026094356	Marina Pereira	Engenharia de Softw...	ativo	nenhuma	2026-10-05T18:38:08.863+00:00	

Figura 5 — Resultado da query mostrando as versões com __START_AT e __END_AT.

Consulta do histórico

```

SELECT matricula, nome, curso, status, bolsa, __START_AT, __END_AT
FROM projeto2.silver.dim_aluno_scd2
WHERE matricula = '<MATRICULA_COM_ALTERACAO>'
ORDER BY __START_AT;

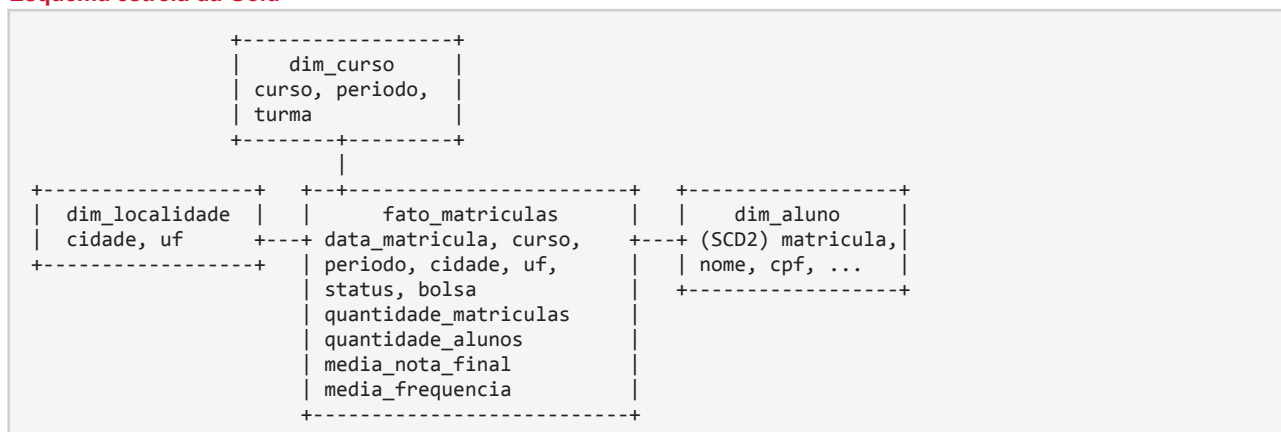
```

6. Publicação (camada Gold)

A Gold foi modelada em **esquema estrela** e publicada como materialized views no schema `projeto2.gold`. Ela roda em um pipeline separado, por isso lê a Silver pelo nome completo (`projeto2.silver...`). Materialized views são recalculadas de forma incremental quando possível, o que deixa as atualizações mais rápidas.

6.1 Modelo dimensional

Esquema estrela da Gold



A `fato_matriculas` é agregada: cada linha representa uma combinação de data de matrícula, curso, período, cidade, UF, status e bolsa. Os relacionamentos com as dimensões são feitos pelas chaves naturais (`curso, periodo, cidade, uf`).

Métrica	Cálculo	Significado
<code>quantidade_matriculas</code>	<code>count(matricula)</code>	Total de matrículas no grupo
<code>quantidade_alunos</code>	<code>countDistinct(cpf)</code>	Alunos distintos no grupo
<code>media_nota_final</code>	<code>round(avg(nota_final), 2)</code>	Nota média do grupo
<code>media_frequencia</code>	<code>round(avg(frequencia), 2)</code>	Frequência média do grupo

6.2 Código

04_projeto2_gold/transformations/camada_gold.py

```
from pyspark import pipelines as dp
from pyspark.sql.functions import count, countDistinct, avg, round as spark_round

SILVER = "projeto2.silver.projeto2_silver_quality"

# =====
# GOLD 1 – DIMENSÃO LOCALIDADE
# =====

@dp.materialized_view(
    name="dim_localidade",
    comment="Dimensão de cidades e UFs dos alunos"
)
def dim_localidade():
    return (
        spark.read.table(SILVER)
        .select("cidade", "uf")
        .filter("cidade IS NOT NULL AND uf IS NOT NULL")
        .dropDuplicates(["cidade", "uf"])
    )
```

```

# =====
# GOLD 2 – DIMENSÃO CURSO
# =====

@dp.materialized_view(
    name="dim_curso",
    comment="Dimensão de cursos, períodos e turmas"
)
def dim_curso():
    return (
        spark.read.table(SILVER)
        .select("curso", "periodo", "turma")
        .filter("curso IS NOT NULL")
        .dropDuplicates(["curso", "periodo", "turma"])
    )

# =====
# GOLD 3 – DIMENSÃO ALUNO (SCD2)
# =====

@dp.materialized_view(
    name="dim_aluno",
    comment="Dimensão de alunos com histórico SCD Tipo 2"
)
def dim_aluno():
    return spark.read.table("projeto2.silver.dim_aluno_scd2")

# =====
# GOLD 4 – FATO MATRÍCULAS
# =====

@dp.materialized_view(
    name="fato_matriculas",
    comment="Matrículas, alunos e desempenho por curso, localidade, status e bolsa"
)
def fato_matriculas():
    return (
        spark.read.table(SILVER)
        .groupBy(
            "data_matricula",
            "curso",
            "periodo",
            "cidade",
            "uf",
            "status",
            "bolsa"
        )
        .agg(
            count("matricula").alias("quantidade_matriculas"),
            countDistinct("cpf").alias("quantidade_alunos"),
            spark_round(avg("nota_final"), 2).alias("media_nota_final"),
            spark_round(avg("frequencia"), 2).alias("media_frequencia")
        )
    )

```

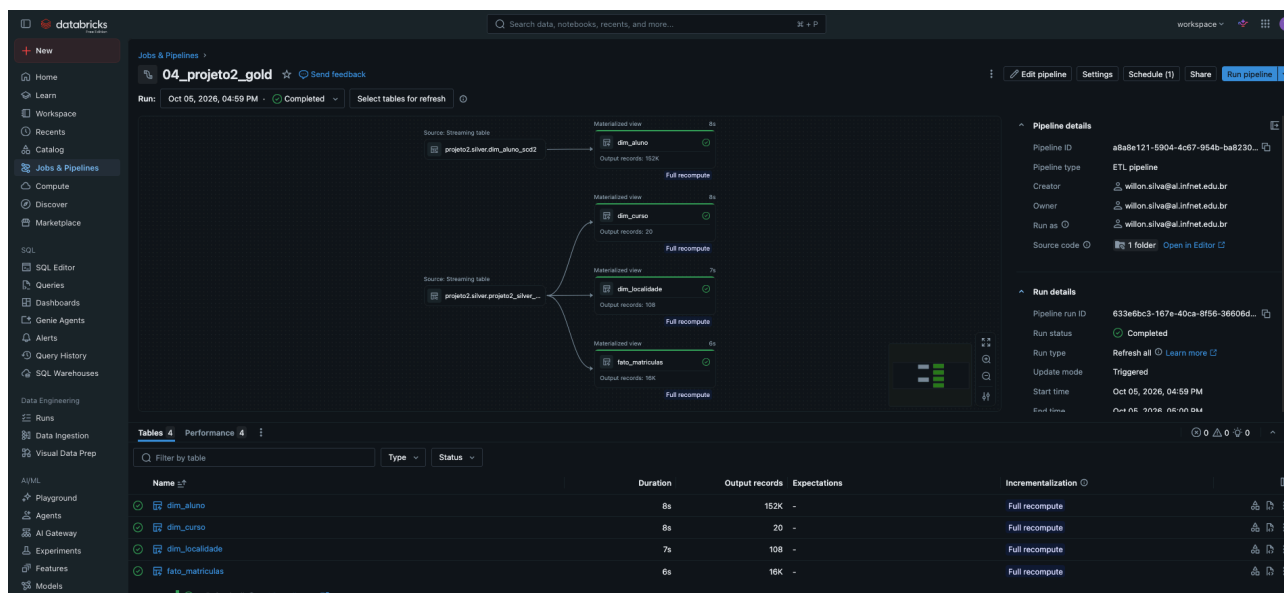


Figura 6 — Grafo do pipeline Gold mostrando as quatro materialized views e as tabelas da Silver como origem externa.

6.3 Consulta na Gold

Exemplo de consulta analítica: matrículas ativas por data, UF e curso, com médias ponderadas.

queries.dbquery.ipynb — Query na Gold

```
-- Matrículas ativas por data, UF e curso:
SELECT
  f.data_matricula,
  f.uf,
  f.curso,
  SUM(f.quantidade_matriculas) AS quantidade_matriculas,
  SUM(f.quantidade_alunos) AS quantidade_alunos,
  ROUND(
    SUM(f.media_nota_final * f.quantidade_matriculas)
    / SUM(f.quantidade_matriculas), 2
  ) AS media_nota_final,
  ROUND(
    SUM(f.media_frequencia * f.quantidade_matriculas)
    / SUM(f.quantidade_matriculas), 2
  ) AS media_frequencia
FROM projeto2.gold.fato_matriculas AS f
WHERE f.status = 'ativo'
GROUP BY
  f.data_matricula,
  f.uf,
  f.curso
ORDER BY
  f.data_matricula DESC,
  quantidade_matriculas DESC;
```

The screenshot displays the Databricks SQL Editor interface. On the left, the 'Tree view: ON' sidebar shows the project structure, including '00-queries' under 'projeto2'. The main editor area contains a SQL query that filters for 'ativo' status and calculates aggregated metrics. Below the query, the 'Table' view shows the results of the query, which are displayed in a table format with columns for data, uf, curso, and various aggregated metrics.

```
84 -- Matrículas ativas por data, UF e curso:
85 SELECT
86   f.data_matricula,
87   f.uf,
88   f.curso,
89   SUM(f.quantidade_matriculas) AS quantidade_matriculas,
90   SUM(f.quantidade_alunos) AS quantidade_alunos,
91   ROUND(
92     SUM(f.media_nota_final * f.quantidade_matriculas)
93     / SUM(f.quantidade_matriculas), 2
94   ) AS media_nota_final,
95   ROUND(
96     SUM(f.media_frequencia * f.quantidade_matriculas)
97     / SUM(f.quantidade_matriculas), 2
98   ) AS media_frequencia
99 FROM projeto2.gold.fato_matriculas AS f
100 WHERE f.status = 'ativo'
101 GROUP BY
102   f.data_matricula,
103   f.uf,
104   f.curso
105 ORDER BY
106   f.data_matricula DESC,
107   quantidade_matriculas DESC;
108
```

	1.1 data_matricula	1.2 uf	1.3 curso	2.1 quantidade_matriculas	2.2 quantidade_alunos	1.2 media_nota_final	1.2 media_frequencia
1	2026-09-30	AC	Ciência de Dados	3	3	2.33	40
2	2026-09-30	MA	Ciência de Dados	3	3	4.13	40
3	2026-09-30	RJ	Ciência de Dados	3	3	5.93	40
4	2026-09-29	AC	Ciência de Dados	24	24	5.37	40
5	2026-09-29	MA	Ciência de Dados	23	23	5.24	40
6	2026-09-29	RJ	Ciência de Dados	23	23	4.85	40

At the bottom of the table view, it indicates '1,000 rows' and '1.431s runtime'. The status bar at the bottom right shows 'Refreshed at 10/5/26, 6:51 PM (Now)'.

Figura 6 — Consulta na camada gold.

7. Esquema de transação (Delta Lake)

Todas as tabelas do projeto são Delta. Cada escrita feita pelo pipeline vira um commit no transaction log (`_delta_log`), o que garante as propriedades ACID: um update parcial nunca fica visível, leituras e escritas concorrentes não se atrapalham e cada versão da tabela fica registrada.

Camada	Operação	Como é registrada no log
Bronze	Append incremental do Auto Loader	STREAMING UPDATE a cada lote processado
Silver (quality, quarentena, validada)	Append incremental do streaming	STREAMING UPDATE
Silver (SCD1 e SCD2)	AUTO CDC (merge por matricula)	Operações de MERGE com linhas inseridas, atualizadas e fechadas
Gold	Refresh das materialized views	Recomputação completa ou incremental, conforme o caso
Todas	Manutenção automática do pipeline	OPTIMIZE e VACUUM executados pelo próprio Databricks

O histórico de transações de cada tabela pode ser consultado com **DESCRIBE HISTORY**:

Histórico de transações

```
DESCRIBE HISTORY projeto2.silver.projeto2_silver_quality;  
DESCRIBE HISTORY projeto2.silver.dim_aluno_scd2;
```

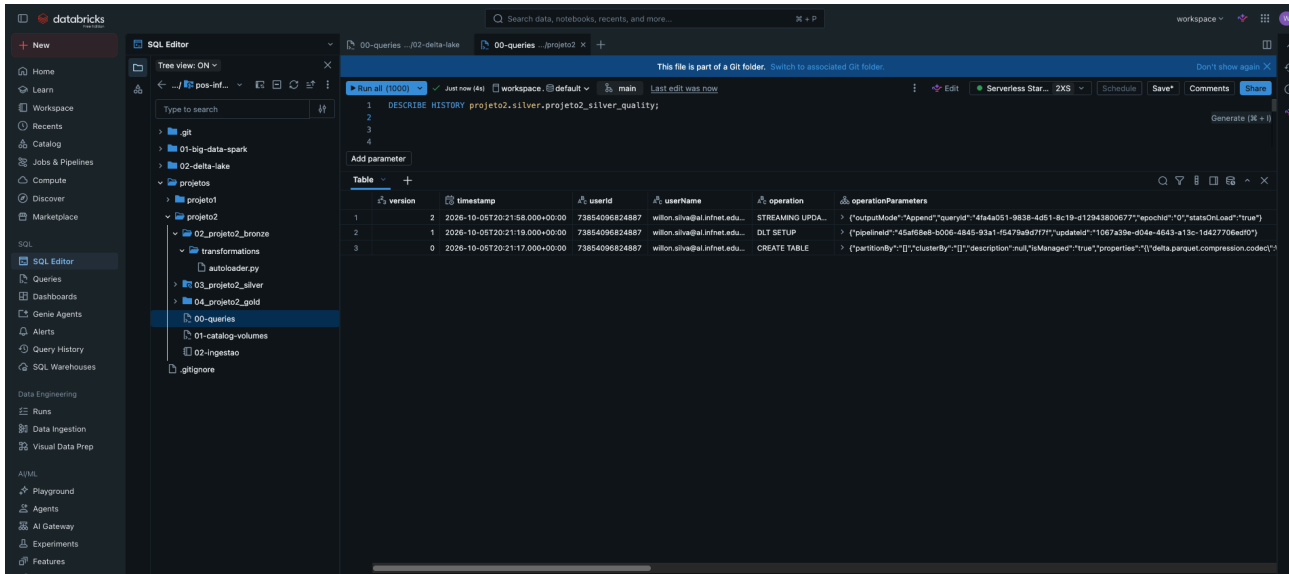
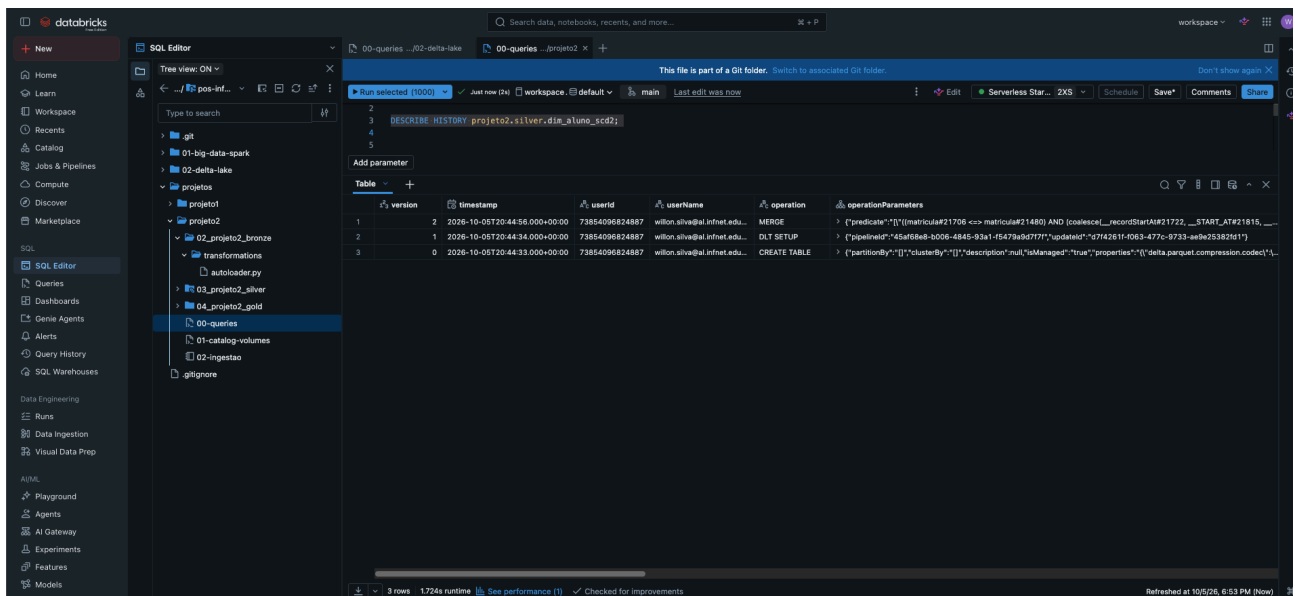


Figura 7 — Histórico de transações da tabela projeto2_silver_quality



version	timestamp	user_id	user_name	operation	operation_parameters
1	2006-10-06T20:44:56.000+00:00	73854096824887	wilson.silva@al.infnet.edu...	MERGE	[{"predicate": "[{matricula#21706 <=> matricula#21480} AND (coalesce(__recordStartMS#11722, __START_AT#21815, __..."}]
2	2006-10-06T20:44:34.000+00:00	73854096824887	wilson.silva@al.infnet.edu...	DLT SETUP	[{"pipelineId": "45af6b68-4006-4845-93a1-f5479a9d777f", "updateId": "6742611-0563-477c-9733-ee9e25387611"}]
3	2006-10-06T20:44:33.000+00:00	73854096824887	wilson.silva@al.infnet.edu...	CREATE TABLE	[{"partitionBy": "[{}]", "clusterBy": "[{}]", "description": null, "isManaged": "true", "properties": "[{}]", "delta.parquet.compression.codec": "z..."}]

Figura 8 — Histórico de transações da tabela dim_aluno_scd2;

8. Configuração do workflow

8.1 Pipelines

Parâmetro	Bronze	Silver	Gold
Pasta raiz	02_projeto2_bronze	03_projeto2_silver	04_projeto2_gold
Catálogo padrão	projeto2	projeto2	projeto2
Schema padrão	bronze	silver	gold
Compute	Serverless	Serverless	Serverless
Modo de execução	Triggered	Triggered	Triggered
Canal	Current	Current	Current

Escolhi **Serverless** porque o ambiente é o Databricks Free Edition, que só oferece esse tipo de compute, e porque ele dispensa o dimensionamento de cluster. Escolhi o modo **Triggered** porque os dados chegam em lotes: o pipeline processa o que há de novo e desliga, o que também evita estourar a cota de compute da conta gratuita (erro `CLUSTER_CREATION_RESOURCE_EXHAUSTED`, que enfrentei durante o desenvolvimento). Em compute tradicional, eu precisaria definir tipo de máquina, número de workers e autoscaling.

8.2 Jobs de orquestração

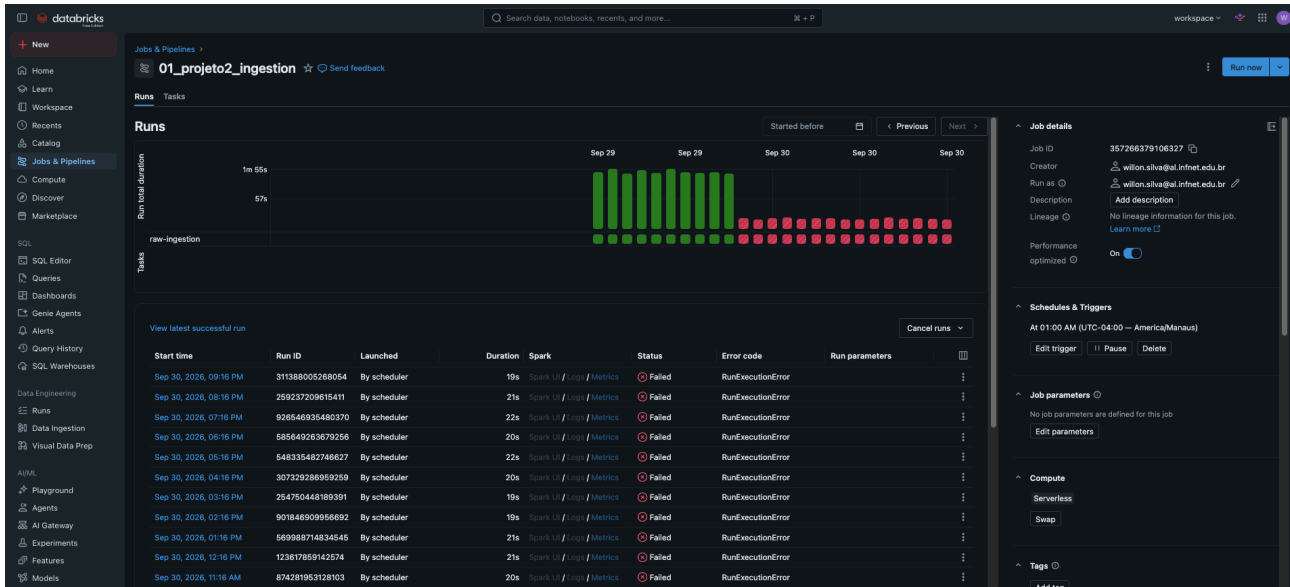


Figura 9 — Orquestração da ingestão;

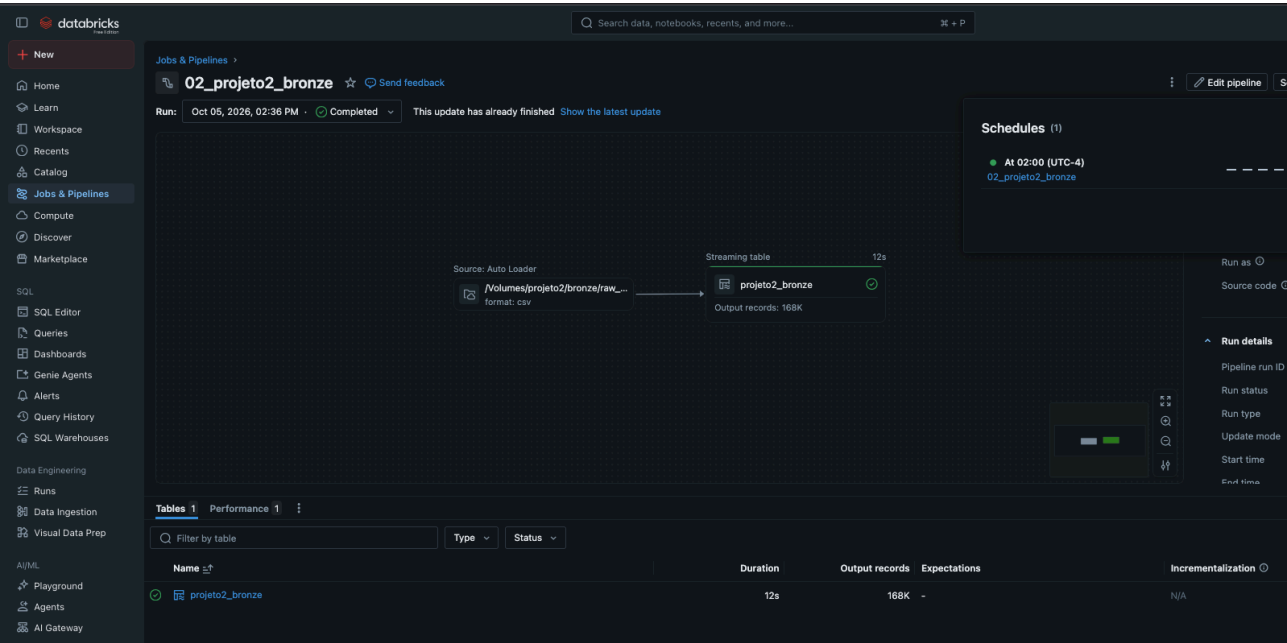


Figura 10 — Orquestração da bronze;

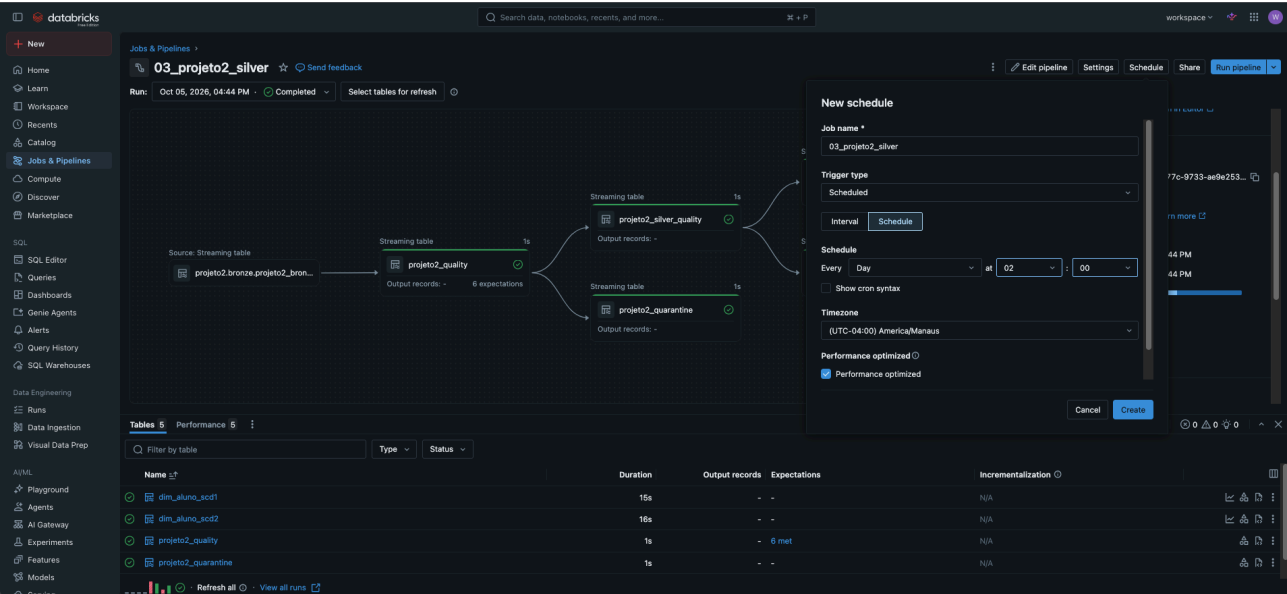


Figura 11 — Orquestração da silver;

The screenshot displays the Databricks interface for a pipeline named **04_projeto2_gold**. The pipeline is in a 'Completed' state, having run on October 05, 2026, at 04:59 PM. The DAG shows a source streaming table **projeto2silver_dim_aluno_acd2** feeding into **dim_aluno** (8s, 152K records). Another source **projeto2silver_dim_curso_acd2** feeds into **dim_curso** (8s, 20 records). A third source **projeto2silver_dim_localidade_acd2** feeds into **dim_localidade** (7s, 108 records). Finally, all three dimension tables feed into the fact table **fato_matriculas** (6s, 16K records). All tables are marked as 'Full recompute'.

Name	Duration	Output records	Expectations	Incrementalization
dim_aluno	8s	152K	-	Full recompute
dim_curso	8s	20	-	Full recompute
dim_localidade	7s	108	-	Full recompute
fato_matriculas	6s	16K	-	Full recompute

On the right, the 'Schedules' section shows a schedule for **04_projeto2_gold** at 02:00 UTC-4. The 'Run details' section shows the pipeline run ID **633e6bc3-167e-40ca-8f56-36606d...**, status **Completed**, and start time **Oct 05, 2026, 04:59 PM**.

Figura 12 — Orquestração da gold;

9. Métricas de qualidade (Data Quality Report)

Como as expectations rodam no modo warn, o pipeline registra no event log, a cada update, quantos registros passaram e falharam em cada regra. Essas métricas aparecem na aba **Data quality** da tabela `projeto2_quality` na interface do pipeline e também podem ser consultadas por SQL.

9.1 Métricas por expectation (event log)

Métricas de qualidade a partir do event log

```
SELECT
  e.dataset,
  e.name AS expectation,
  SUM(e.passed_records) AS aprovados,
  SUM(e.failed_records) AS reprovados,
  ROUND(100 * SUM(e.failed_records)
    / (SUM(e.passed_records) + SUM(e.failed_records)), 2) AS pct_falha
FROM (
  SELECT explode(
    from_json(
      details:flow_progress:data_quality:expectations,
      'array<struct<name: string, dataset: string,
        passed_records: bigint, failed_records: bigint>>'
    )
  ) AS e
  FROM event_log(TABLE(projeto2.silver.projeto2_quality))
  WHERE event_type = 'flow_progress'
)
GROUP BY e.dataset, e.name
ORDER BY reprovados DESC;
```

9.2 Volume da quarentena

Análise da quarentena

```
SELECT
  (SELECT COUNT(*) FROM projeto2.silver.projeto2_silver_quality) AS aprovados,
  (SELECT COUNT(*) FROM projeto2.silver.projeto2_quarantine) AS quarentena;

-- Motivo da reprovação (um registro pode falhar em mais de uma regra)
SELECT
  SUM(CASE WHEN matricula IS NULL THEN 1 ELSE 0 END) AS sem_matricula,
  SUM(CASE WHEN nome IS NULL THEN 1 ELSE 0 END) AS sem_nome,
  SUM(CASE WHEN cpf IS NULL THEN 1 ELSE 0 END) AS sem_cpf,
  SUM(CASE WHEN email IS NULL THEN 1 ELSE 0 END) AS sem_email,
  SUM(CASE WHEN curso IS NULL THEN 1 ELSE 0 END) AS sem_curso,
  SUM(CASE WHEN NOT coalesce(status IN ('ativo','trancado','formado','jubilado'), false)
    THEN 1 ELSE 0 END) AS status_invalido
FROM projeto2.silver.projeto2_quarantine;
```

9.3 Análise

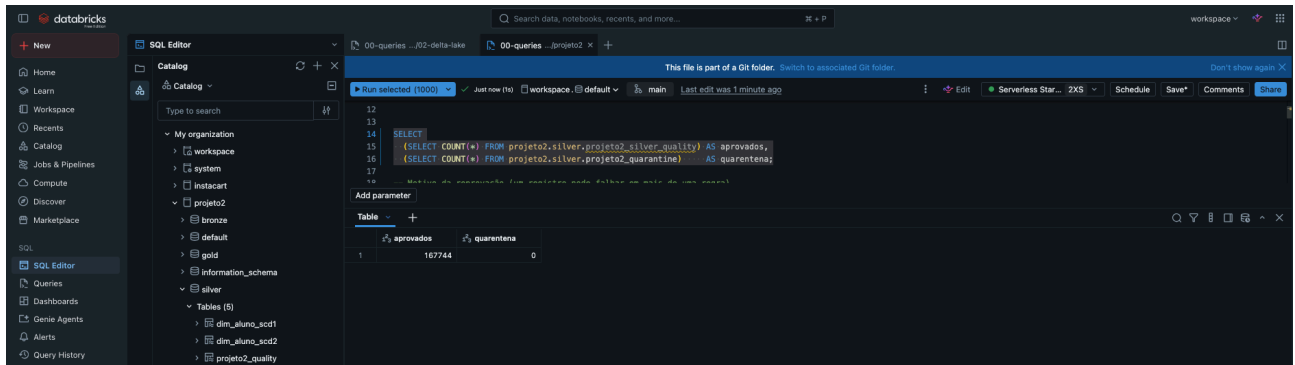


Figura 13 — Contagem em quarentena e aprovados;

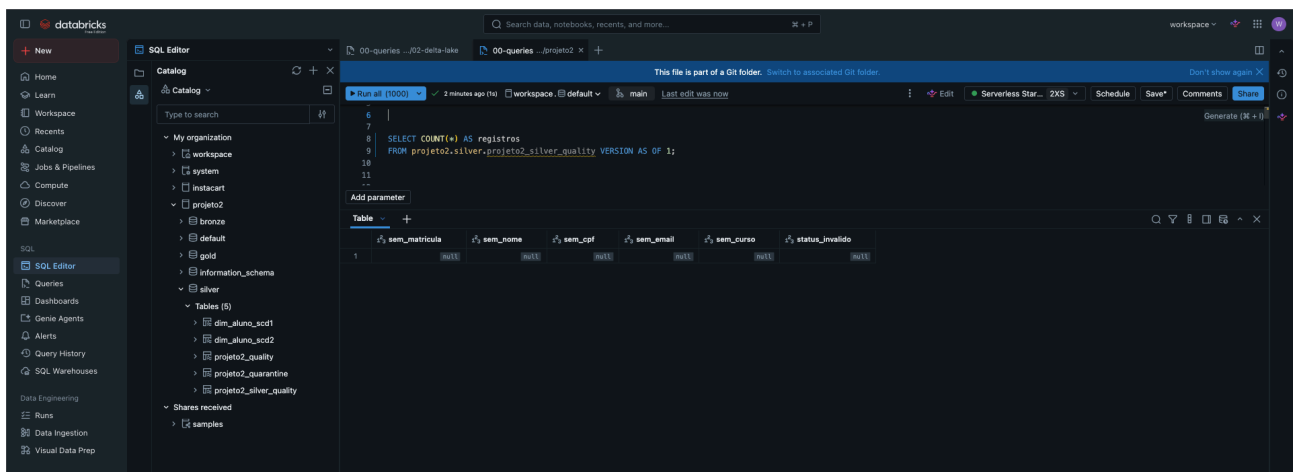


Figura 14 — Motivo da reprovação;

10. Governança e privacidade

10.1 Controle de acesso

O acesso à Gold é dado ao grupo **analista_dados**. O grupo recebe permissão de uso no catálogo e no schema e permissão de leitura nas tabelas da Gold. As camadas Bronze e Silver não são liberadas, então o analista só enxerga os dados já tratados. Exemplos de queries abaixo.

queries.dbquery.ipynb — Permissões

```
GRANT USE CATALOG
ON CATALOG projeto2
TO `analista_dados`;

GRANT USE SCHEMA
ON SCHEMA projeto2.gold
TO `analista_dados`;

GRANT SELECT
ON SCHEMA projeto2.gold
TO `analista_dados`;

GRANT SELECT
ON TABLE projeto2.gold.fato_matriculas
TO `analista_dados`;

GRANT SELECT
ON TABLE projeto2.gold.dim_aluno
TO `analista_dados`;

GRANT SELECT
ON TABLE projeto2.gold.dim_curso
TO `analista_dados`;

GRANT SELECT
ON TABLE projeto2.gold.dim_localidade
TO `analista_dados`;
```

10.2 Classificação e mascaramento

Para a privacidade, criei a governed tag **classification** com três níveis (**green**, **yellow** e **red**) e classifiquei as colunas da **dim_aluno**. Depois criei uma função de máscara e uma política (ABAC) no schema Gold: toda coluna com tag **red** é mascarada para o grupo **analista_dados**, sem precisar configurar tabela por tabela.

Coluna	Classificação	Mascarada para analista_dados?
cpf	red	Sim
email	yellow	Não
nome	yellow	Não
matricula	green	Não

queries.dbquery.ipynb — Governança

```
CREATE GOVERNED TAG classification
DESCRIPTION 'Classificação de sensibilidade dos dados'
VALUES ('green', 'yellow', 'red');

ALTER TABLE projeto2.gold.dim_aluno
ALTER COLUMN cpf
SET TAGS ('classification' = 'red');
```

```
ALTER TABLE projeto2.gold.dim_aluno
ALTER COLUMN email
SET TAGS ('classification' = 'yellow');

ALTER TABLE projeto2.gold.dim_aluno
ALTER COLUMN nome
SET TAGS ('classification' = 'yellow');

ALTER TABLE projeto2.gold.dim_aluno
ALTER COLUMN matricula
SET TAGS ('classification' = 'green');

ALTER TABLE projeto2.gold.dim_aluno
ALTER COLUMN matricula
SET TAGS ('classification' = 'green');

CREATE OR REPLACE FUNCTION projeto2.gold.mask_red(cpf STRING)
RETURNS STRING
RETURN '***REDACTED***';

CREATE OR REPLACE FUNCTION projeto2.gold.mask_red(email STRING)
RETURNS STRING
RETURN '***REDACTED***';

CREATE POLICY mask_red_columns
ON SCHEMA projeto2.gold
COLUMN MASK projeto2.gold.mask_red
TO `analista_dados`
FOR TABLES
MATCH COLUMNS has_tag_value('classification', 'red') AS m
ON COLUMN m;
```

11. Linhagem no Unity Catalog

O Unity Catalog captura a linhagem automaticamente a partir das execuções dos pipelines e das queries. Isso permite ver, para cada tabela e coluna, de onde o dado veio e quem o consome, o que ajuda em análise de impacto e auditoria. No projeto, a linhagem completa vai do volume `raw_csv` até a `fato_matriculas`.

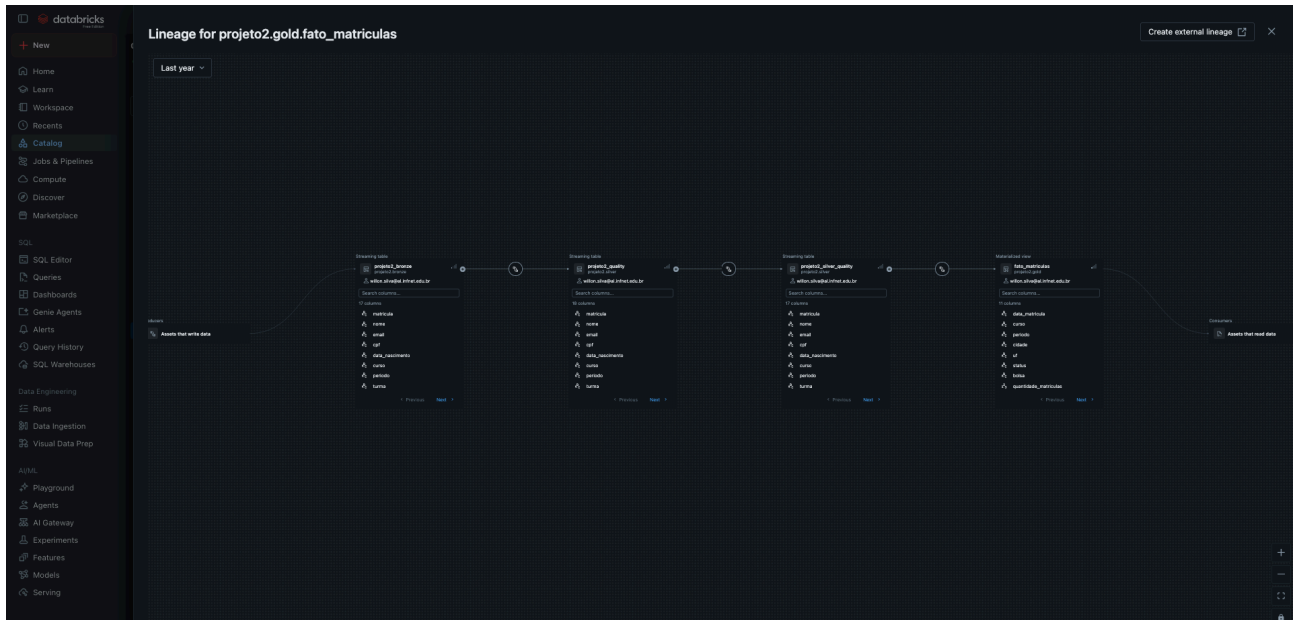


Figura 15 — Exemplo de linhagem de dados: tabela `fato_matriculas`;

12. Otimização, performance e monitoramento

12.1 Layout físico

- **Particionamento:** a `projeto2_silver_quality` é particionada por `uf` e `curso`, as colunas mais usadas como filtro nas consultas.
- **Z-Order:** a mesma tabela usa `pipelines.autoOptimize.zOrderCols = "cpf,cidade"`, aplicado na manutenção automática do pipeline. O Z-Order organiza os dados dentro dos arquivos para que filtros por CPF e cidade leiam menos arquivos (data skipping).

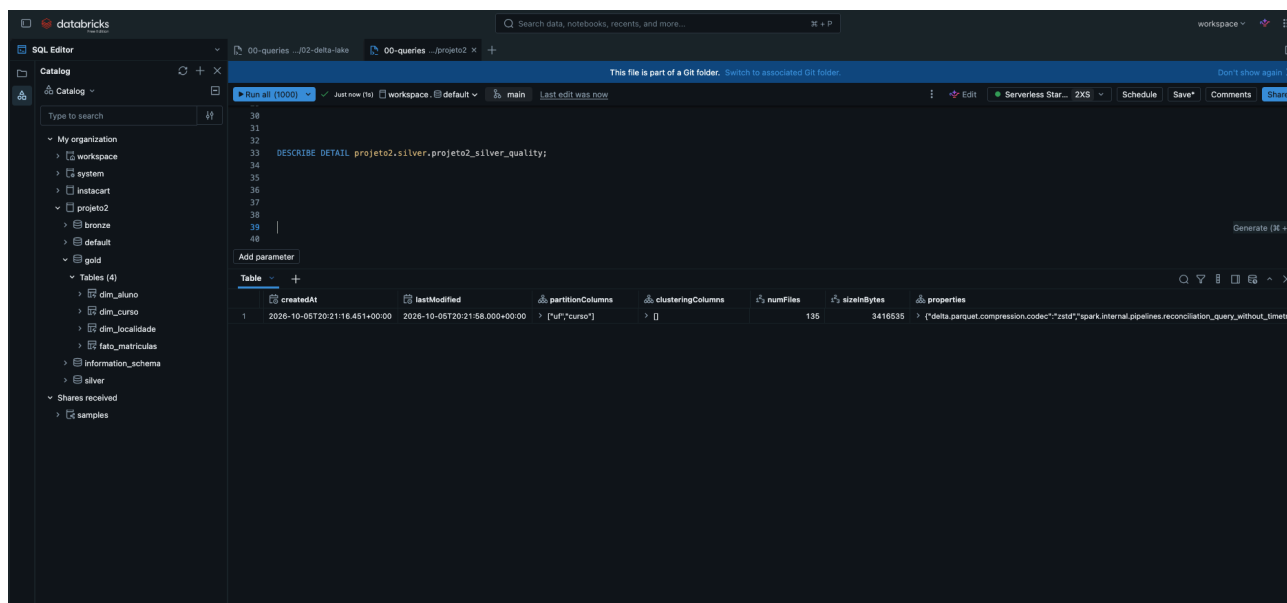


Figura 16 — Propriedades de otimização da tabela `projeto2_silver_quality`;

12.2 Monitoramento

Para acompanhar a execução, uso três fontes: a interface do pipeline (grafo com contagem de registros e status em tempo real durante o update), o event log de cada pipeline e as tabelas de sistema dos Jobs. A query abaixo, do `queries`, mostra o resultado das execuções dos últimos 7 dias:

`queries.dbquery.ipynb` — Monitoramento dos jobs

```
SELECT
    result_state,
    COUNT(*) AS quantidade
FROM system.lakeflow.job_run_timeline
WHERE period_start_time >= current_timestamp() - INTERVAL 7 DAYS
GROUP BY result_state
ORDER BY quantidade DESC;
```

The screenshot displays the Databricks SQL Editor interface. On the left, the 'Catalog' sidebar shows a tree view of the workspace, including 'projeto2' and its subfolders like 'bronze', 'default', 'gold', and 'silver'. The main editor area contains a SQL script with the following content:

```
106 TO 'analista_dados'
107 FOR TABLES
108 MATCH COLUMNS has_tag_value('classification', 'red') AS m
109 ON COLUMN m;
110
111 ---- MONITORAMENTO DOS JOBS ----
112
113 SELECT
114     result_state,
115     COUNT(*) AS quantidade
116 FROM system.lakeflow.job_run_timeline
117 WHERE period_start_time >= current_timestamp() - INTERVAL 7 DAYS
118 GROUP BY result_state
119 ORDER BY quantidade DESC;
120
121 ---- QUERY NA GOLD ----
122
123 -- Matrículas ativas por data, UF e curso:
124 SELECT
125     f.data_matricula,
126     f.uf,
```

Below the query, a table titled 'Table' shows the results of the first query:

	result_state	quantidade
1	SUCCEEDED	37
2	ERROR	27

Figura 17 — Monitoramento dos jobs;